UNIVERSIDAD AMERICANA · UAM

INFORME FINAL

Laboratorio de concurrencia en Java y JavaFX

Oficina con dos ventanillas que atienden solicitudes en hilos propios, protegen un contador compartido y se detienen de forma cooperativa sin bloquear la interfaz gráfica.

Asignatura	CNU0525 · Programación de Aplicaciones de Escritorio
Grupo	1
Docente	José Durán García
Integrantes	Daniella Rocha Kellys Bellanger Nora Obregón Daris Mayorga
Repositorio	github.com/kellys-bellanger/Laboratorio_Hilos
Fecha	26 de octubre de 2026

CNU0525 · PROGRAMACIÓN DE APLICACIONES DE ESCRITORIO · GRUPO 1

Informe: Laboratorio de concurrencia en Java y JavaFX

Profesor: José Durán García · **Equipo:** Daniella Rocha (líder), Kellys Bellanger, Nora Obregón, Daris Mayorga
Repositorio: github.com/kellys-bellanger/Laboratorio_Hilos

La aplicación simula una oficina con dos ventanillas que atienden solicitudes en hilos propios y suman a un contador compartido. La ventana JavaFX permite iniciar y detener la atención sin congelarse. Las capturas, la tabla de pruebas y el análisis por estación están en el documento *Evidencias y análisis*.

1. ¿Qué error apareció antes de sincronizar?

Una **condición de carrera con pérdida de actualizaciones**. `total++` no es atómico: son tres pasos (leer, sumar, escribir). Si los dos hilos leen el mismo valor, ambos escriben el mismo resultado y un incremento se pierde. Con 1 000 incrementos por hilo, como pide el enunciado, apareció en **1 de 10 corridas** (1 540 en lugar de 2 000). Con 1 000 casi no se ve porque el primer hilo termina antes de que arranque el segundo, así que repetimos con 100 000 por hilo: **3 de 10 corridas** perdieron conteos (142 109, 199 878 y 188 217 de 200 000). Con `synchronized` y con `AtomicInteger` todas las corridas dieron el total exacto.

2. ¿Cómo evitó bloquear JavaFX?

- El trabajo (`sleep` de 250 ms por solicitud) corre en `Hilo-Ventanilla-1` y `Hilo-Ventanilla-2`, nunca en el *JavaFX Application Thread*.
- El `join()` lo hace un hilo aparte, `Hilo-Supervisor`; los eventos de los botones nunca llaman a `join()` ni a `sleep()`.
- Los controles solo se modifican en el hilo de JavaFX: un `Timeline` refresca la vista cada 100 ms leyendo el contador, los estados y una cola de mensajes (`ConcurrentLinkedQueue`), y el aviso de fin llega con `Platform.runLater()`. Así hay como máximo 10 actualizaciones por segundo, sin importar cuántos mensajes generen los hilos (reto de diseño).

3. ¿Qué condiciones verificó para detener y reiniciar?

- **Detener:** `detener()` baja la bandera `volatile activo` y llama a `interrupt()`, que despierta el `sleep()`. El hilo captura `InterruptedException`, restaura la interrupción y sale del ciclo por sí mismo, sin `Thread.stop()`. El estado pasa por *Deteniendo* → *Detenido*.
- **Reiniciar:** *Iniciar* sigue deshabilitado hasta que `Hilo-Supervisor` confirma con `join()` que **ambos** hilos terminaron; además `iniciar()` revisa `hayHilosVivos()`. Cada sesión crea hilos nuevos. Al cerrar la ventana también se pide la detención cooperativa.
- Lo comprueban la captura del estado *Detenido* ("Hilos vivos: no") y las pruebas JUnit `detenerInterrumpeElSleepYTerminaDetenida` y `elAvisoDeFinSoloLlegaCuandoNoQuedanHilosVivos` (32 pruebas en total, todas aprobadas).

4. ¿Qué mejoraría para procesar 100 000 solicitudes?

- Usar un **pool de hilos** (`ExecutorService`) o hilos virtuales de Java 21 en vez de un hilo por ventanilla.
- Cambiar el contador a `AtomicInteger` o `LongAdder`, que evitan el bloqueo de `synchronized` cuando hay mucha contención.
- **No registrar cada solicitud** en el área de texto: mostrar resúmenes o solo las últimas N líneas, porque un texto enorme vuelve lenta la interfaz.
- Mantener la actualización agrupada y usar `javafx.concurrent.Task` con `updateProgress()` para la barra de progreso.